Zadanie Cząstkowe 3: Implementacja w Kubernetes i Sprawozdanie z Realizacji Testów

Przedmiot: Bezpieczeństwo CI/CD / Zaawansowane Środowiska Chmurowe i Konteneryzacja
Temat: 3 – System zbierania metryk i monitoringu (Prometheus + Grafana + własny eksporter w Pythonie) **Data wykonania zrzutów i testów:** 2026-11-05 (Minikube z GPU NVIDIA RTX 4080 SUPER + GitHub Actions)

Linki do projektu (repozytorium publiczne)

Zasób	Adres
B. Repozytorium z manifestami	https://github.com/Shengan01/Inference-Server
Manifesty Kubernetes (`base` , `overlays` , `addons` , `gitops` , `tests` , `examples`)	https://github.com/Shengan01/Inference-Server/tree/main/k8s
README repozytorium	https://github.com/Shengan01/Inference-Server/blob/main/README.md
GitHub Actions (potok CI/CD, wszystkie przebiegi)	https://github.com/Shengan01/Inference-Server/actions
Definicja potoku	https://github.com/Shengan01/Inference-Server/blob/main/.github/workflows/ci-cd.yaml
Przykładowy przebieg z raportem testów (artefakt `test-report`)	https://github.com/Shengan01/Inference-Server/actions/runs/41239646305
Obraz kontenera (GitHub Container Registry)	https://github.com/Shengan01/Inference-Server/pkgs/container/mcp-agent
Instrukcja uruchomienia i testów	https://github.com/Shengan01/Inference-Server/blob/main/docs/INSTRUKCJA_URUCHOMIENIA_I_TESTOW.md

Zawartość sprawozdania. Część **A** (rozdz. 1–3): potwierdzenie wdrożenia w klastrze (polecenia + zrzuty), prezentacja testów z Zadania 2 (cel, polecenia, zrzuty, wyniki) oraz komentarz do funkcjonalności. Część **B** (rozdz. 4): link do repozytorium z manifestami i opis jego zawartości. Zrzuty z terminala przedstawiają rzeczywisty wynik poleceń uruchomionych na klastrze (generowane skryptem `scripts/capture_evidence.py`), a zrzuty interfejsów (Prometheus, Grafana, Alertmanager, GitHub Actions) są zrzutami z przeglądarki.

Część A

1. Potwierdzenie wdrożenia w klastrze Kubernetes

Środowisko: jednowęzłowy klaster Minikube (sterownik Docker, Kubernetes v1.35) z dostępem do **GPU NVIDIA GeForce RTX 4080 SUPER (16 GB)**, namespace `inference-monitoring`. Serwer inferencji to **prawdziwy Triton z backendem vLLM** (obraz `tritonserver:24.06-vllm-python-py3`) obsługujący model Qwen2-1.5B (GPTQ) na GPU. System wdrożono dwiema równoważnymi ścieżkami: skryptem lokalnym oraz GitOps (ArgoCD synchronizuje klaster z repozytorium).

```
./scripts/deploy_minikube_gpu.sh                              # klaster Minikube z GPU + wdrożenie lokalne (overlay local-gpu)
kubectl apply -f k8s/gitops/argocd-application-gpu.yaml    # wdrożenie GitOps (overlay gpu z gałęzi main)
```

1.1. Mapa wymaganych obiektów

Obiekt	Konfiguracja w projekcie	Manifest	Zrzut
StatefulSet	`prometheus`: stała tożsamość, `volumeClaimTemplates` (2Gi), retencja 15 dni, probes, `securityContext`	`k8s/base/prometheus/statefulset.yaml`	1.3
Deployment	`grafana`, `alertmanager`, `mcp-agent` (eksporter), `triton-server` (Triton + vLLM, zasób `nvidia.com/gpu`); probes (w tym `startupProbe` dla ładowania modelu), strategia `Recreate` (PVC RWO, jedno GPU)	`k8s/base/*/deployment.yaml`, `k8s/overlays/gpu/`	1.3
Service	5 × `ClusterIP` (Prometheus, Alertmanager, Grafana, Triton, MCP Agent), nazwane porty	`k8s/base/*/service.yaml`	1.4
Ingress	NGINX, TLS, host `inference.local`; ścieżki `/grafana`, `/v2`, `/mcp` (rewrite)	`k8s/base/ingress/ingress.yaml`	1.4
PVC	`prometheus-storage-prometheus-0` 2Gi (z szablonu StatefulSet), `grafana-storage` 1Gi, `model-repository` 5Gi (wspólny dla Tritona i agenta; PV `hostPath`)	`k8s/base/grafana/pvc.yaml`, StatefulSet, `k8s/overlays/gpu/model-storage.yaml`	1.5
ConfigMap	konfiguracja Prometheusa i Alertmanagera, reguły alertów, datasource i dashboard Grafany, `config.pbtxt` i `model.json` modelu	`k8s/base/*/configmap*.yaml`, `k8s/overlays/gpu/model-json-configmap.yaml`	1.6
Secret	`grafana-secret` (Opaque), `inference-tls` (`kubernetes.io/tls`); tworzone skryptem, bez wartości w Git; wzór w repozytorium	`scripts/bootstrap_secrets.sh`, `k8s/examples/secrets.example.yaml`	1.6
NetworkPolicy	`default-deny-all` + 6 reguł dla każdej pary komunikujących się usług	`k8s/base/network-policy.yaml`	1.7
Requests i limits	jawne `requests` i `limits` CPU/pamięć w każdym kontenerze (także init-container)	manifesty workloadów	1.8
ResourceQuota	4 CPU / 8Gi (requests), 8 CPU / 16Gi (limits), 5 PVC, 15 podów	`k8s/base/resource-quota.yaml`	1.8
LimitRange	domyślne requests/limits kontenera oraz zakres 500Mi–10Gi dla PVC	`k8s/base/resource-quota.yaml`	1.8

1.2. Klaster i namespace

```
kubectl get nodes -o wide && kubectl get namespaces
```

Klaster i namespace

```
$ kubectl get nodes -o wide && kubectl get namespaces
NAME   STATUS   ROLES           AGE   VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                       KERNEL-VERSION           CONTAINER-RUNTIME
gpu    Ready    control-plane   14m   v1.35.1   192.168.58.2   <none>        Debian GNU/Linux 12 (bookworm)   7.2.6-76070206-generic   docker://29.2.1
NAME                  STATUS   AGE
argocd                Active   8m1s
default               Active   14m
inference-monitoring  Active   11m
ingress-nginx         Active   14m
kube-node-lease       Active   14m
kube-public           Active   14m
kube-system           Active   14m
monitoring            Active   3m19s
```

1.3. Deployment i StatefulSet

```
kubectl -n inference-monitoring get deployments,statefulsets,pods
```

Deployment / StatefulSet / Pod

```
$ kubectl -n inference-monitoring get deployments,statefulsets,pods
NAME                              READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/alertmanager      1/1     1            1           11m
deployment.apps/grafana           1/1     1            1           11m
deployment.apps/mcp-agent         1/1     1            1           11m
deployment.apps/triton-server     1/1     1            1           11m

NAME                            READY   AGE
statefulset.apps/prometheus     1/1     11m

NAME                                  READY   STATUS    RESTARTS   AGE
pod/alertmanager-7448b9bdc-c5784      1/1     Running   0          11m
pod/grafana-db86b4cc6-klxsb           1/1     Running   0          11m
pod/mcp-agent-785cbf648f-9kvjq        1/1     Running   0          3m59s
pod/prometheus-0                      1/1     Running   0          11m
pod/triton-server-594f9bfb5c-tscsv    1/1     Running   0          11m
```

Komentarz: Prometheus działa jako StatefulSet (dane szeregów czasowych na trwałym wolumenie), pozostałe komponenty jako Deployment. Wszystkie pody są w stanie `Running 1/1`.

Serwer inferencji korzysta z GPU: węzeł udostępnia Kubernetesowi zasób `nvidia.com/gpu` (wtyczka urządzeń NVIDIA), a pod Tritona widzi kartę:

```
kubectl describe node | grep nvidia.com/gpu
kubectl -n inference-monitoring exec deploy/triton-server -c triton-server -- nvidia-smi --
query-gpu=name,memory.used,memory.total --format=csv
```

GPU w klastrze: zasób nvidia.com/gpu i karta widoczna w podzie Tritona

```
$ kubectl describe node | grep -E 'nvidia.com/gpu' ; echo; kubectl -n inference-monitoring exec deploy/triton-server -c triton-server -- nvidia-smi --query-gpu=name,memory.used,memory.total
  --format=csv
  nvidia.com/gpu:     1
  nvidia.com/gpu:     1
  nvidia.com/gpu     1           1

name, memory.used [MiB], memory.total [MiB]
NVIDIA GeForce RTX 4080 SUPER, 9430 MiB, 16376 MiB
```

Model jest załadowany w prawdziwym Triton (backend vLLM) i generuje tekst:

```
kubectl -n inference-monitoring exec deploy/triton-server -c triton-server -- curl -s -X POST
localhost:8000/v2/models/finance-qwen/generate -d '{"text_input":"...","parameters":
{"stream":false,"max_tokens":24}}'
```

Prawdziwy Triton + vLLM: model załadowany i generuje tekst

```
$ kubectl -n inference-monitoring exec deploy/triton-server -c triton-server -- sh -c "curl -s localhost:8000/v2/health/ready -o /dev/null -w 'server ready: HTTP %{http_code}\n'; curl -s loc
  alhost:8000/v2/models/finance-qwen/ready -o /dev/null -w 'model finance-qwen ready: HTTP %{http_code}\n'; curl -s -X POST localhost:8000/v2/models/finance-qwen/generate -d '{\"text_input\"
  :\"Analyze Q3 tech market earnings.\",\"parameters\":{\"stream\":false,\"temperature\":0,\"max_tokens\":24}}'; echo"
server ready: HTTP 200
model finance-qwen ready: HTTP 200
{"model_name":"finance-qwen","model_version":"1","text_output":"Analyze Q3 tech market earnings. I'm sorry, but I'm unable to provide the requested information as it pertains to a hypothetic
  al scenario involving a specific"}
```

1.4. Service i Ingress

```
kubectl -n inference-monitoring get services,ingress
IP=$(minikube ip); curl -sk --resolve inference.local:443:$IP
https://inference.local/grafana/api/health
```

Service i Ingress

```
$ kubectl -n inference-monitoring get services,ingress
NAME                           TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
service/alertmanager-service   ClusterIP   10.97.53.17     <none>        9093/TCP                     11m
service/grafana-service        ClusterIP   10.99.15.250    <none>        3000/TCP                     11m
service/mcp-agent-service      ClusterIP   10.103.91.214   <none>        5000/TCP                     11m
service/prometheus-service     ClusterIP   10.96.250.143   <none>        9090/TCP                     11m
service/triton-service         ClusterIP   10.106.63.25    <none>        8000/TCP,8001/TCP,8002/TCP   11m

NAME                                                CLASS   HOSTS             ADDRESS        PORTS     AGE
ingress.networking.k8s.io/inference-monitoring-ingress       nginx   inference.local   192.168.58.2   80, 443   11m
ingress.networking.k8s.io/inference-monitoring-mcp-ingress   nginx   inference.local   192.168.58.2   80, 443   11m
```

Dostęp zewnętrzny przez Ingress (TLS, kody odpowiedzi i przekierowanie HTTP → HTTPS):

Dostęp przez Ingress (TLS, przekierowanie HTTP -> HTTPS)

```
$ IP=$(minikube ip); for p in grafana/api/health v2/health/ready mcp/health; do echo "https://inference.local/$p -> $(curl -sk --resolve inference.local:443:$IP https://inference.local/$p -o
   /dev/null -w '%{http_code}')"; done; echo "http://inference.local/grafana/ -> $(curl -s --resolve inference.local:80:$IP http://inference.local/grafana/ -o /dev/null -w '%{http_code}') (p
  rzekierowanie na HTTPS)"
https://inference.local/grafana/api/health -> 200
https://inference.local/v2/health/ready -> 200
https://inference.local/mcp/health -> 200
http://inference.local/grafana/ -> 308 (przekierowanie na HTTPS)
```

1.5. PersistentVolumeClaim

```
kubectl -n inference-monitoring get pvc && kubectl get pv
```

PersistentVolumeClaim i PersistentVolume

```
$ kubectl -n inference-monitoring get pvc && kubectl get pv
NAME                            STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS       VOLUMEATTRIBUTESCLASS   AGE
grafana-storage                 Bound    pvc-5b7f4099-5f02-4b22-8186-236796a4ceb0   1Gi        RWO            standard           <unset>                 11m
model-repository                Bound    model-repository-pv                        5Gi        RWO            model-repository   <unset>                 11m
prometheus-storage-prometheus-0   Bound    pvc-ef7b2022-756c-4f3d-b290-78d762cec524   2Gi        RWO            standard           <unset>                 11m
NAME                                CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                                                                   STORAGECLASS       VOLUME
  ATTRIBUTESCLASS   REASON   AGE
model-repository-pv                 5Gi        RWO            Retain           Bound    inference-monitoring/model-repository                                   model-repository   <unset
  >                         11m
pvc-5b7f4099-5f02-4b22-8186-236796a4ceb0   1Gi        RWO            Delete           Bound    inference-monitoring/grafana-storage                                    standard           <unset
  >                         11m
pvc-a78d7fd7-444f-4a63-9a6e-78bcb4a28f29   2Gi        RWO            Delete           Bound    monitoring/prometheus-kps-prometheus-db-prometheus-kps-prometheus-0     standard           <unset
  >                         2m53s
pvc-ef7b2022-756c-4f3d-b290-78d762cec524   2Gi        RWO            Delete           Bound    inference-monitoring/prometheus-storage-prometheus-0                    standard           <unset
  >                         11m
```

1.6. ConfigMap i Secret

```
kubectl -n inference-monitoring get configmaps,secrets
```

ConfigMap i Secret (nazwy i typy, bez wartości)

```
$ kubectl -n inference-monitoring get configmaps,secrets
NAME                                  DATA   AGE
configmap/alertmanager-config         1      11m
configmap/grafana-dashboards          2      11m
configmap/grafana-datasources         1      11m
configmap/kube-root-ca.crt            1      11m
configmap/loadgen-script              1      9m29s
configmap/prometheus-alerting-rules   1      11m
configmap/prometheus-config           1      11m
configmap/triton-model-config         1      11m
configmap/triton-model-json           1      11m

NAME                     TYPE                DATA   AGE
secret/grafana-secret    Opaque              2      11m
secret/inference-tls     kubernetes.io/tls   2      11m
```

Komentarz: wartości sekretów celowo nie są pokazywane ani przechowywane w Git. `grafana-secret` zawiera losowe hasło administratora, `inference-tls` certyfikat self-signed dla `inference.local`; oba tworzy `scripts/bootstrap_secrets.sh`. Wzór obiektów (z wartościami zastępczymi) znajduje się w `k8s/examples/secrets.example.yaml`.

1.7. NetworkPolicy

```
kubectl -n inference-monitoring get networkpolicies
```

NetworkPolicy

```
$ kubectl -n inference-monitoring get networkpolicies
NAME                              POD-SELECTOR                                   AGE
allow-alertmanager                app.kubernetes.io/name=alertmanager            11m
allow-grafana                     app.kubernetes.io/name=grafana                 11m
allow-kube-dns                    <none>                                         11m
allow-loadgen-to-prometheus       app.kubernetes.io/name=prometheus              9m29s
allow-mcp-agent                   app.kubernetes.io/name=mcp-agent               11m
allow-operator-prometheus-scrape  app.kubernetes.io/name in (mcp-agent,triton-server)   3m20s
allow-prometheus-scrape           app.kubernetes.io/name=prometheus              11m
allow-triton-server               app.kubernetes.io/name=triton-server           11m
default-deny-all                  <none>                                         11m
loadgen-egress                    app.kubernetes.io/name=loadgen                 9m29s
```

Komentarz: `default-deny-all` blokuje cały ruch w namespace, a pozostałe polityki otwierają wyłącznie wymagane połączenia (Prometheus → Triton/agent/Alertmanager, Grafana → Prometheus, agent → Prometheus/Triton, ruch z Ingress do usług, DNS). Polityki `loadgen-*` pochodzą z Jobu testowego (rozdz. 2).

1.8. Requests, limits, ResourceQuota i LimitRange

```
kubectl -n inference-monitoring get pods -o custom-
columns=POD:.metadata.name,CONTAINER:.spec.containers[*].name,CPU_REQ:.spec.containers[*].resou
rces.requests.cpu,CPU_LIM:.spec.containers[*].resources.limits.cpu,MEM_REQ:.spec.containers[*].
resources.requests.memory,MEM_LIM:.spec.containers[*].resources.limits.memory
kubectl -n inference-monitoring describe resourcequota inference-monitoring-quota
kubectl -n inference-monitoring describe limitrange inference-monitoring-limits
```

Requests i limits każdego kontenera

```
$ kubectl -n inference-monitoring get pods -o custom-columns=POD:.metadata.name,CONTAINER:.spec.containers[*].name,CPU_REQ:.spec.containers[*].resources.requests.cpu,CPU_LIM:.spec.containers
  [*].resources.limits.cpu,MEM_REQ:.spec.containers[*].resources.requests.memory,MEM_LIM:.spec.containers[*].resources.limits.memory
POD                          CONTAINER       CPU_REQ   CPU_LIM   MEM_REQ   MEM_LIM
alertmanager-7448b9bdc-c5784 alertmanager    50m       200m      64Mi      128Mi
grafana-db86b4cc6-klxsb      grafana         100m      500m      128Mi     512Mi
mcp-agent-785cbf648f-9kvjq   mcp-agent       100m      500m      128Mi     512Mi
prometheus-0                 prometheus      200m      1         256Mi     1Gi
triton-server-594f9bfb5c-tscsv triton-server 2         4         4Gi       12Gi
```

ResourceQuota

```
$ kubectl -n inference-monitoring describe resourcequota inference-monitoring-quota
Name:                   inference-monitoring-quota
Namespace:              inference-monitoring
Resource                Used     Hard
--------                ----     ----
limits.cpu              6200m    8
limits.memory           14464Mi  16Gi
persistentvolumeclaims  3        5
pods                    5        15
requests.cpu            2450m    4
requests.memory         4672Mi   8Gi
```

LimitRange

```
$ kubectl -n inference-monitoring describe limitrange inference-monitoring-limits
Name:                  inference-monitoring-limits
Namespace:             inference-monitoring
Type                   Resource  Min    Max   Default Request  Default Limit  Max Limit/Request Ratio
----                   --------  ---    ---   ---------------  -------------  -----------------------
Container              cpu       -      -     100m             1              -
Container              memory    -      -     256Mi            1Gi            -
PersistentVolumeClaim  storage   500Mi  10Gi  -                -              -
```

1.9. GitOps i profil operatora

```
kubectl -n argocd get applications && kubectl -n monitoring get pods
```

GitOps: aplikacje ArgoCD i profil operatora

```
$ kubectl -n argocd get applications && kubectl -n monitoring get pods
NAME                      SYNC STATUS   HEALTH STATUS
inference-monitoring-app  Synced        Healthy
kube-prometheus-stack     Synced        Healthy
operator-stack-resources  Synced        Healthy
NAME                                                     READY   STATUS    RESTARTS   AGE
alertmanager-kps-alertmanager-0                          2/2     Running   0          2m56s
kps-operator-f4ccdcbf5-w95n4                             1/1     Running   0          3m8s
kube-prometheus-stack-grafana-69d8d49768-plc5n           3/3     Running   0          3m8s
kube-prometheus-stack-kube-state-metrics-6975ffc467-2bz7h  1/1     Running   0          3m8s
kube-prometheus-stack-prometheus-node-exporter-wdltx     1/1     Running   0          3m8s
prometheus-kps-prometheus-0                              2/2     Running   0          2m56s
```

1.10. Dodatkowe obiekty i zasoby

Obiekt	Uzasadnienie
`ServiceAccount` ×5, `ClusterRole` + `ClusterRoleBinding`	RBAC najmniejszych uprawnień: osobne konta dla każdego komponentu, Prometheus tylko do odczytu (bez dostępu do Secrets)
`Namespace` z etykietami Pod Security	`enforce: baseline`, `audit` / `warn: restricted`
`securityContext` + probes	`runAsNonRoot`, `drop: ALL`, `seccomp: RuntimeDefault`, `readinessProbe` / `livenessProbe` w każdym komponencie
`PersistentVolume` (hostPath) + zasób `nvidia.com/gpu` + `startupProbe`	wspólny wolumen modelu dla Tritona i agenta; GPU przydzielane jako zasób rozszerzony przez wtyczkę urządzeń NVIDIA; `startupProbe` chroni przed zabiciem poda w trakcie ładowania modelu
`Job` + `NetworkPolicy` testowe	generator obciążenia działa wewnątrz klastra (bez narzutu `port-forward`)
`ServiceMonitor` ×2, `PrometheusRule` (CRD)	profil Prometheus Operator, wykrywanie celów przez zasoby Kubernetes
Helm (`kube-prometheus-stack`)	gotowy stos operatora wdrażany z własnymi wartościami
ArgoCD `Application` ×3	GitOps: automatyczna synchronizacja, `prune`, `selfHeal`
Kustomize (`base`, 5 overlayów)	jedna baza, profile `cpu-demo` / `gpu` / `local-*`

Obiekt	Uzasadnienie
GitHub Actions + Trivy + kubeconform	walidacja i skan manifestów oraz obrazu, test na `kind`, automatyczne wdrożenie obrazu

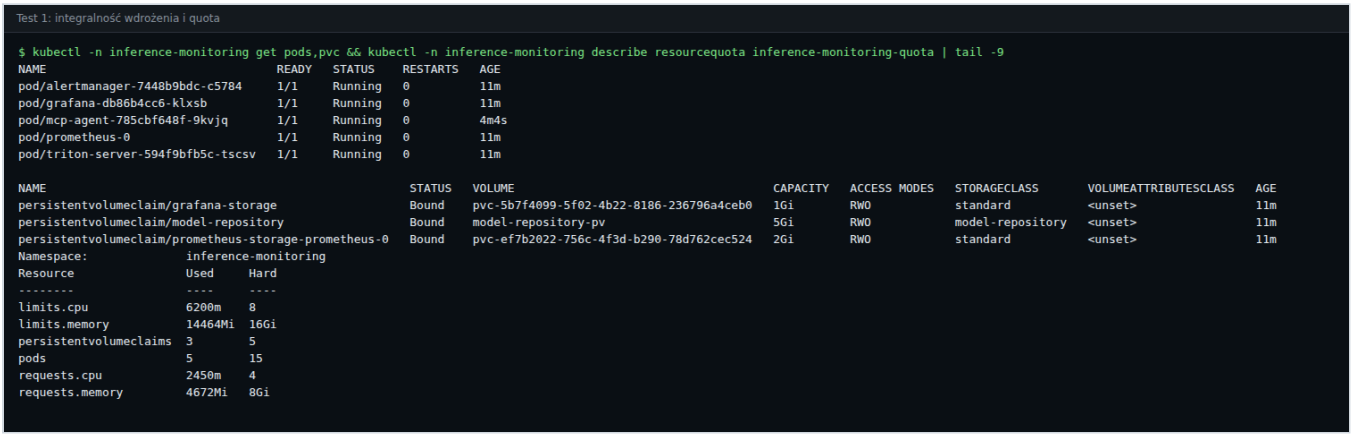
2. Prezentacja testów

Testy odpowiadają procedurze zaplanowanej w Zadaniu 2 (`docs/ZADANIE_2_OPIS_I_ARCHITEKTURA.md`, rozdz. 3, testy 1–8). Poniżej: cel (lub odniesienie), użyte polecenia, zrzut i wynik.

Test 1: integralność wdrożenia i quota

Cel: wszystkie obciążenia działają, PVC są `Bound`, zużycie mieści się w quota (Zadanie 2, Test 1).

```
kubectl -n inference-monitoring get pods,pvc
kubectl -n inference-monitoring describe resourcequota inference-monitoring-quota | tail -9
```



Wynik: 5 podów `Running 1/1`, oba PVC `Bound`, zużycie quota poniżej limitów.

Test 2: audyt bezpieczeństwa (RBAC, NetworkPolicy, non-root)

Cel: uprawnienia najmniejszego zakresu i bezpieczne kontenery (Zadanie 2, Test 2).

```
./scripts/verify_security.sh
```

Test 2: audyt bezpieczeństwa

```
$ ./scripts/verify_security.sh
============================================================
AUDYT BEZPIECZEŃSTWA KUBERNETES DLA CI/CD
Namespace: inference-monitoring
============================================================

[1] Weryfikacja RBAC (Zasada najmniejszych uprawnień - Least Privilege)...
Czy prometheus-sa może czytać pody? (Oczekiwane: yes):
yes
Czy prometheus-sa może tworzyć lub usuwać pody? (Oczekiwane: no):
no
Czy prometheus-sa może czytać Secrets? (Oczekiwane: no):
no

[2] Weryfikacja SecurityContext w podach (Non-root, Capabilities drop)...
Pod: alertmanager-7448b9bdc-c5784
  NonRoot: true
  User: 65534
Pod: grafana-db86b4cc6-klxsb
  NonRoot: true
  User: 472
Pod: mcp-agent-785cbf648f-9kvjq
  NonRoot: true
  User: 10001
Pod: prometheus-0
  NonRoot: true
  User: 65534
Pod: triton-server-594f9bfb5c-tscsv
  NonRoot: false
  User:

[3] Weryfikacja NetworkPolicies (Zero Trust)...
NAME                          POD-SELECTOR                                  AGE
allow-alertmanager            app.kubernetes.io/name=alertmanager           11m
allow-grafana                 app.kubernetes.io/name=grafana                11m
allow-kube-dns                <none>                                        11m
allow-loadgen-to-prometheus   app.kubernetes.io/name=prometheus             9m34s
allow-mcp-agent               app.kubernetes.io/name=mcp-agent              11m
allow-operator-prometheus-scrape   app.kubernetes.io/name in (mcp-agent,triton-server)   3m25s
allow-prometheus-scrape       app.kubernetes.io/name=prometheus             11m
allow-triton-server           app.kubernetes.io/name=triton-server          11m
default-deny-all              <none>                                        11m
loadgen-egress                app.kubernetes.io/name=loadgen                9m34s
Czy w repozytorium/klastrze nie ma jawnego hasla Grafany w manifestach? (Oczekiwane: brak trafien)

[4] Weryfikacja ResourceQuota i LimitRange...
NAME                     REQUEST                                                                                      LIMIT                                                AGE
inference-monitoring-quota   persistentvolumeclaims: 3/5, pods: 5/15, requests.cpu: 2450m/4, requests.memory: 4672Mi/8Gi   limits.cpu: 6200m/8, limits.memory: 14464Mi/16Gi   11m
NAME                      CREATED AT
inference-monitoring-limits   2026-11-05T22:24:49Z

============================================================
Audyt bezpieczeństwa zakończony.
============================================================
```

Wynik: `prometheus-sa` może czytać pody (`yes`), nie może ich usuwać (`no`) ani czytać Secrets (`no`); wszystkie pody mają `NonRoot: true`; polityki sieciowe, ResourceQuota i LimitRange są aktywne.

Test 3: potok metryk i wizualizacji

Cel: Prometheus zbiera metryki z Tritona i eksportera, Alertmanager jest podłączony, Grafana ma wczytane źródło danych i dashboard (Zadanie 2, Test 3).

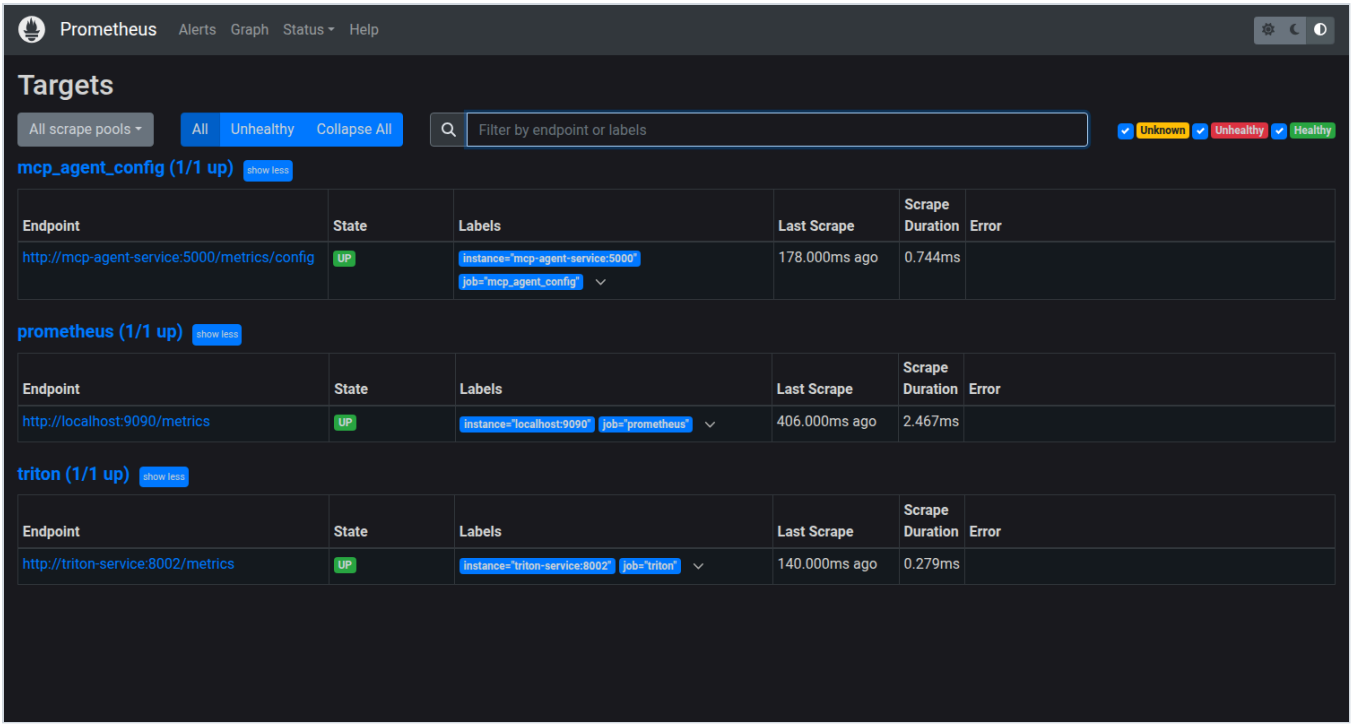
```
curl -s localhost:9090/api/v1/targets | python3 -c "..."      # cele i ich stan
curl -s localhost:9090/api/v1/alertmanagers
```

Test 3: cele Prometheusa i Alertmanager

```
$ curl -s localhost:9090/api/v1/targets | python3 -c "import sys,json; [print(t['scrapePool'], t['health']) for t in json.load(sys.stdin)['data']['activeTargets']]"; echo; curl -s localhost:
  9090/api/v1/alertmanagers | python3 -c "import sys,json; print('alertmanagers:', len(json.load(sys.stdin)['data']['activeAlertmanagers']))"
mcp_agent_config up
prometheus up
triton up

alertmanagers: 1
```

Interfejs Prometheusa (Status → Targets):

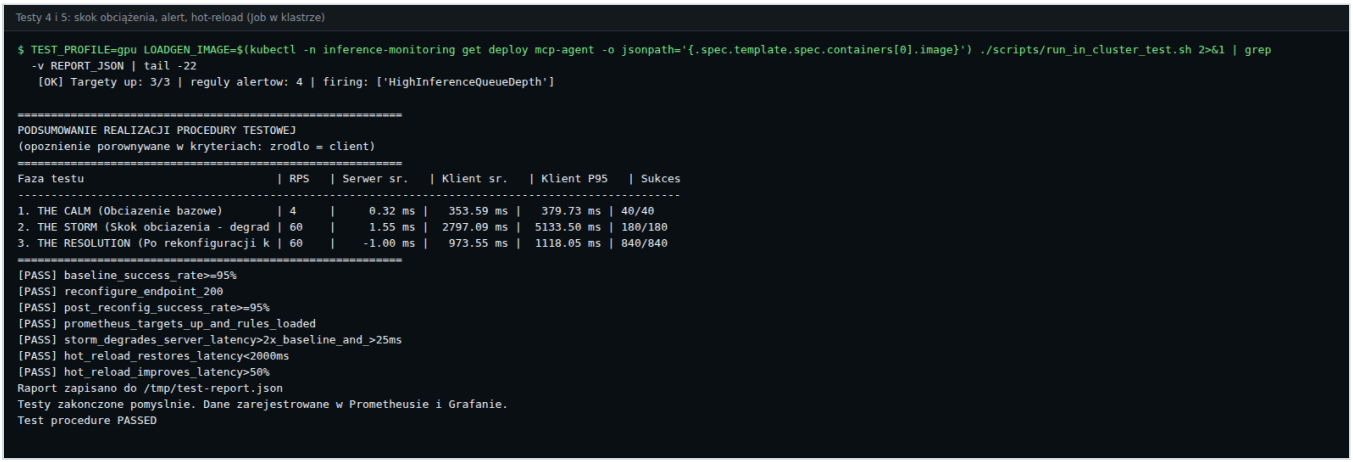


Wynik: Cele `prometheus`, `triton` i `mcp_agent_config` mają stan `up`, Alertmanager jest podłączony.

Testy 4 i 5: skok obciążenia, alert, hot-reload i powrót do normy

Cel: pod wpływem obciążenia rośnie opóźnienie; agent SRE zwiększa `max_num_seqs` (pojemność batcha silnika vLLM) bez przestoju usługi, a opóźnienie wraca do wartości bliskiej bazowej (Zadanie 2, Testy 4 i 5). Test działa jako Job wewnątrz klastra i wysyła prawdziwe żądania generowania tekstu (`/generate`, 64 tokeny) do Tritona z vLLM na GPU. Silnik startuje z zachowawczym `max_num_seqs = 4`; w fazie STORM wysyłanych jest 60 równoczesnych żądań na sekundę, a rekonfiguracja ustawia `max_num_seqs = 512`.

```
TEST_PROFILE=gpu LOADGEN_IMAGE=<obraz mcp-agent> ./scripts/run_in_cluster_test.sh
```

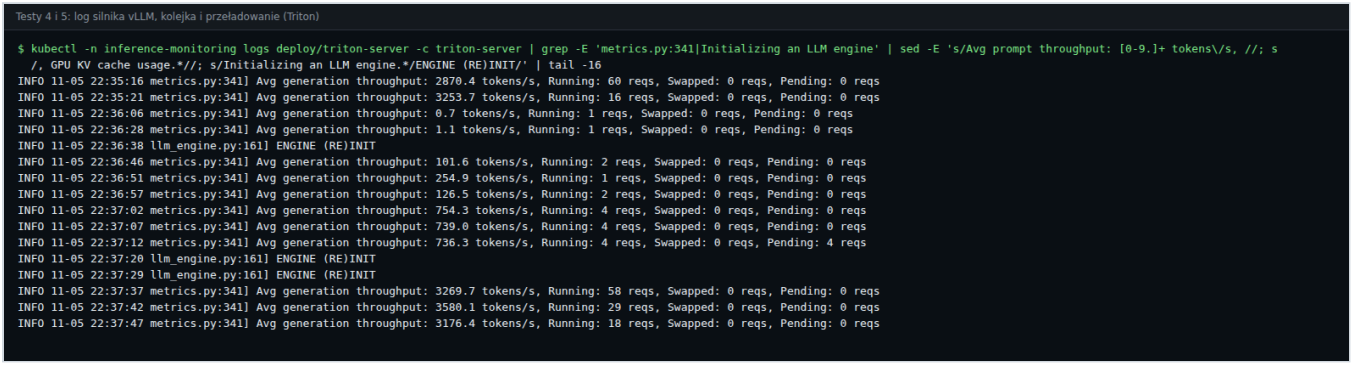


Faza	Żądań/s	Wysłane	Pomyślne	Śr. opóźnienie end-to-end [ms]
1. CALM	4	40	40	354
2. STORM (`max_num_seqs` = 4)	60	180	180	**2797**

Faza	Żądań/s	Wysłane	Pomyślne	Śr. opóźnienie end-to-end [ms]
3. RESOLUTION (po hot-reload, `max_num_seqs` = 512)	60	840	840	**974**

Zachowanie silnika widać w logu vLLM (kolumny `running` i `pending`): w fazie STORM silnik obsługuje 4 żądania naraz, a kolejne czekają w kolejce (`pending` > 0); po przeładowaniu z nowym `max_num_seqs` pracuje na kilkudziesięciu żądaniach jednocześnie i kolejka znika:

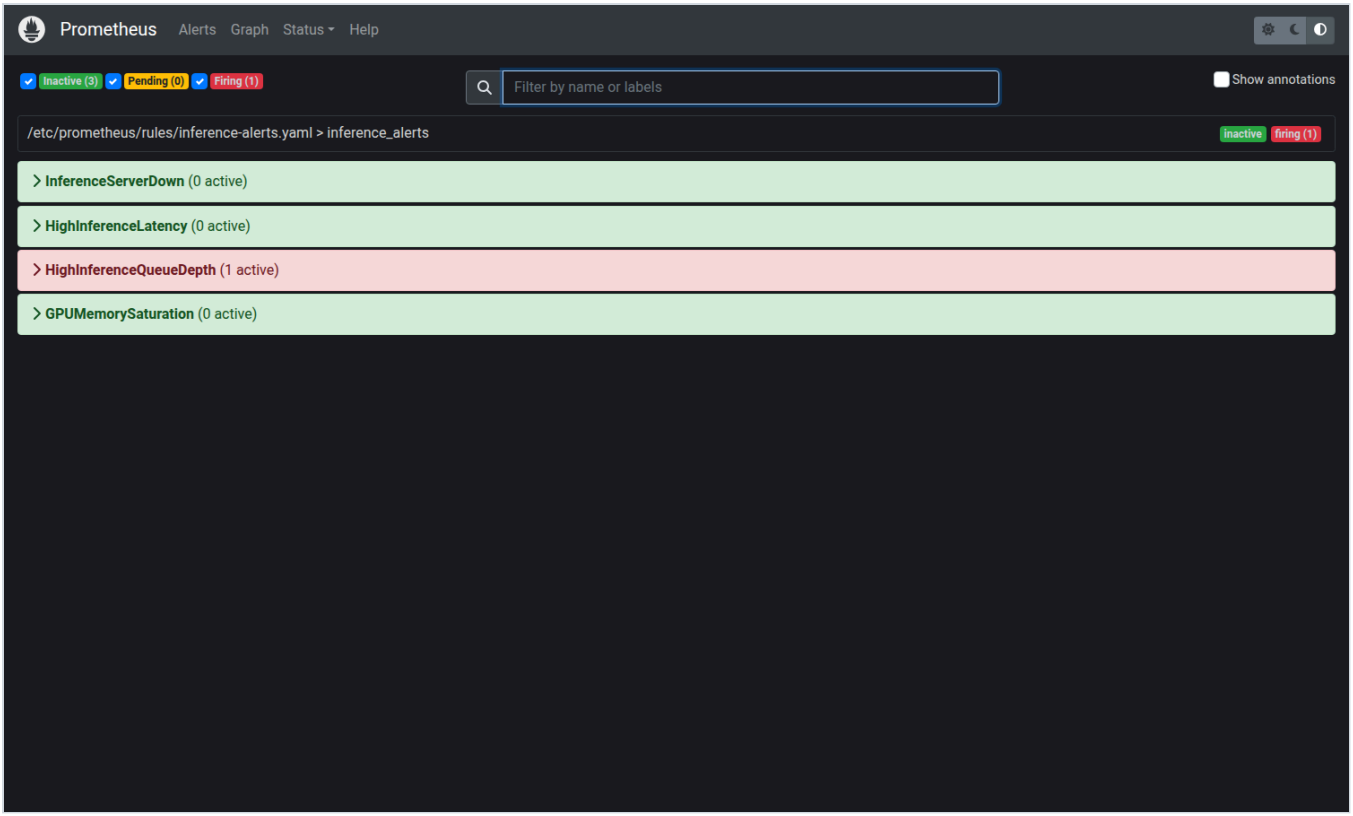
```
kubectl -n inference-monitoring logs deploy/triton-server -c triton-server | grep -E
'metrics.py:341|Initializing an LLM engine'
```

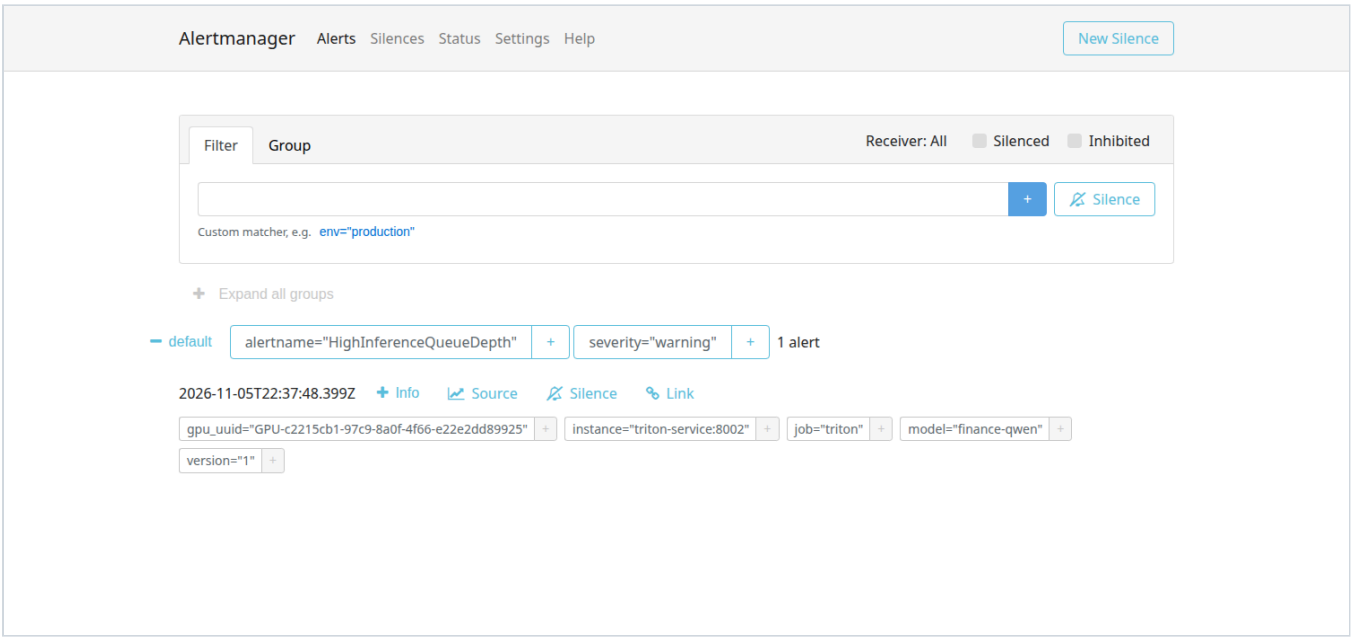


Dashboard Grafany po teście (dwa kolejne przebiegi testu; wykresy GPU, przepustowości i kolejki):



Alerty w Prometheusie i w Alertmanagerze:



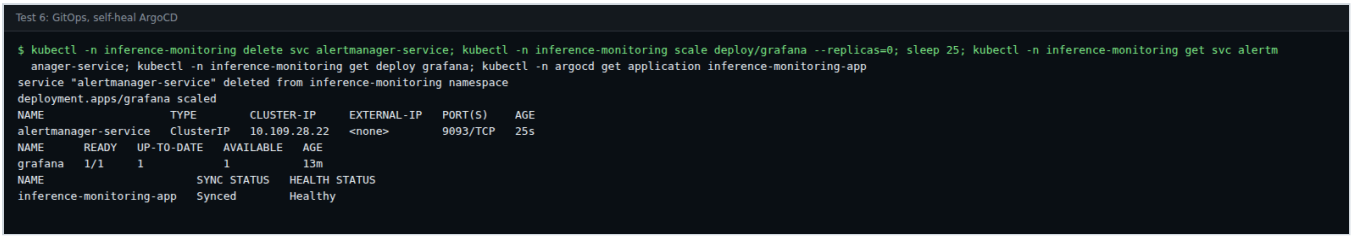


Wynik: wszystkie 7 kryteriów spełnionych. Skuteczność żądań 100%, opóźnienie rośnie w burzy ponad 7.9× względem bazowego, a po hot-reload spada do 974 ms (około 2.9× mniej niż w burzy). Opóźnienie jest mierzone end-to-end wewnątrz klastra, ponieważ model vLLM jest „decoupled”: Triton nie raportuje czasu generowania w `nv_inference_request_duration_us`, a w tej wersji obrazu (24.06) nie ma metryk vLLM. Test jest powtarzalny (trzy kolejne przebiegi dały zbliżone wyniki).

Test 6: GitOps, self-heal ArgoCD

Cel: ręczne zmiany w klastrze są cofane do stanu z repozytorium (Zadanie 2, Test 6).

```
kubectl -n inference-monitoring delete svc alertmanager-service
kubectl -n inference-monitoring scale deploy/grafana --replicas=0
sleep 25; kubectl -n inference-monitoring get svc alertmanager-service; kubectl -n inference-monitoring get deploy grafana
kubectl -n argocd get application inference-monitoring-app
```



Wynik: Usunięty Service i wyzerowane repliki Grafany zostały przywrócone przez ArgoCD, aplikacja ma stan `Synced`.

Test 7: profil Prometheus Operator (ServiceMonitor, Helm)

Cel: Prometheus z chartu `kube-prometheus-stack` wykrywa cele przez ServiceMonitory i ładuje reguły z `PrometheusRule` (Zadanie 2, Test 7).

```
kubectl -n monitoring port-forward svc/kps-prometheus 19090:9090 &
curl -s localhost:19090/api/v1/targets | python3 -c "..."      # cele inference-monitoring
curl -s localhost:19090/api/v1/rules | python3 -c "..."        # grupa inference_alerts
```

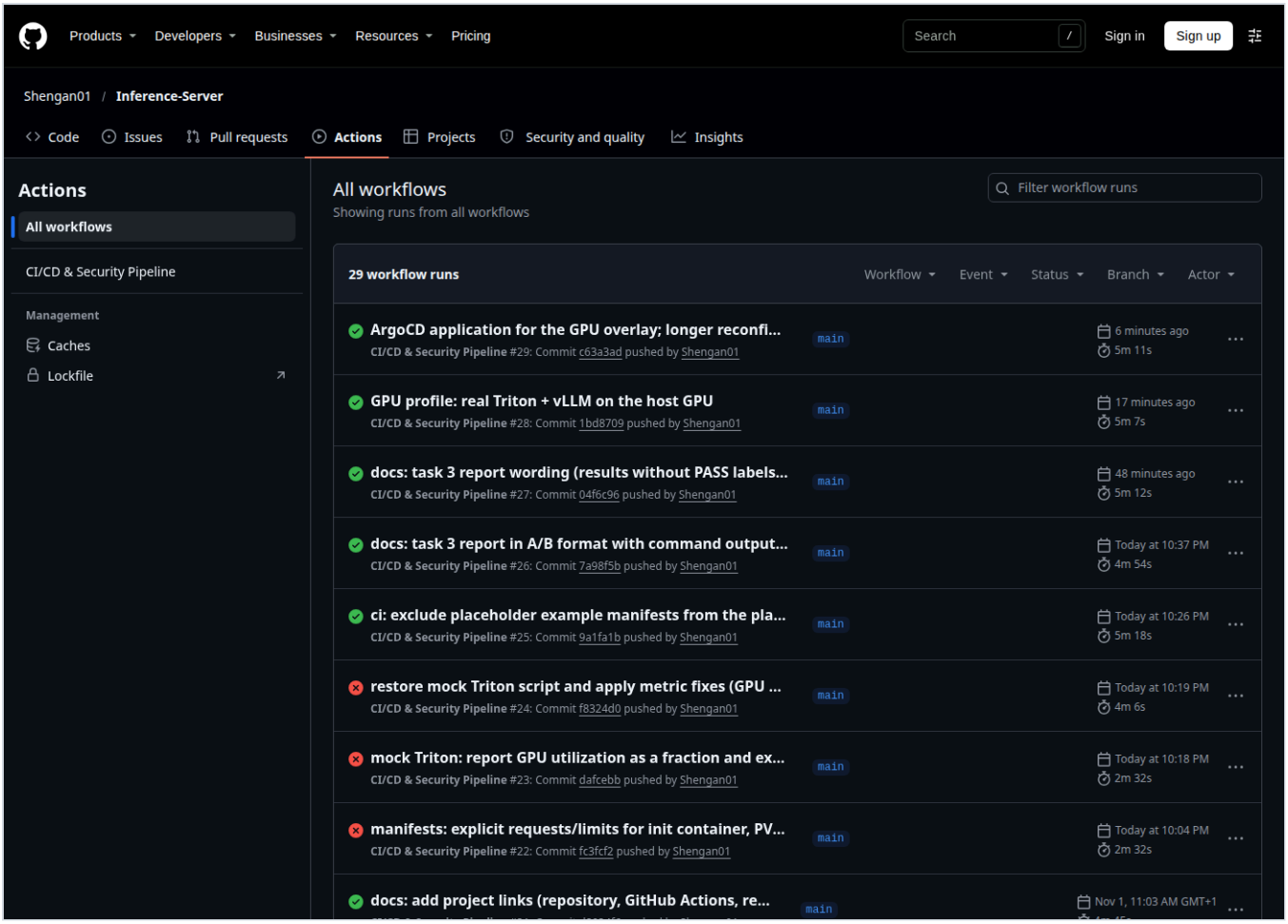


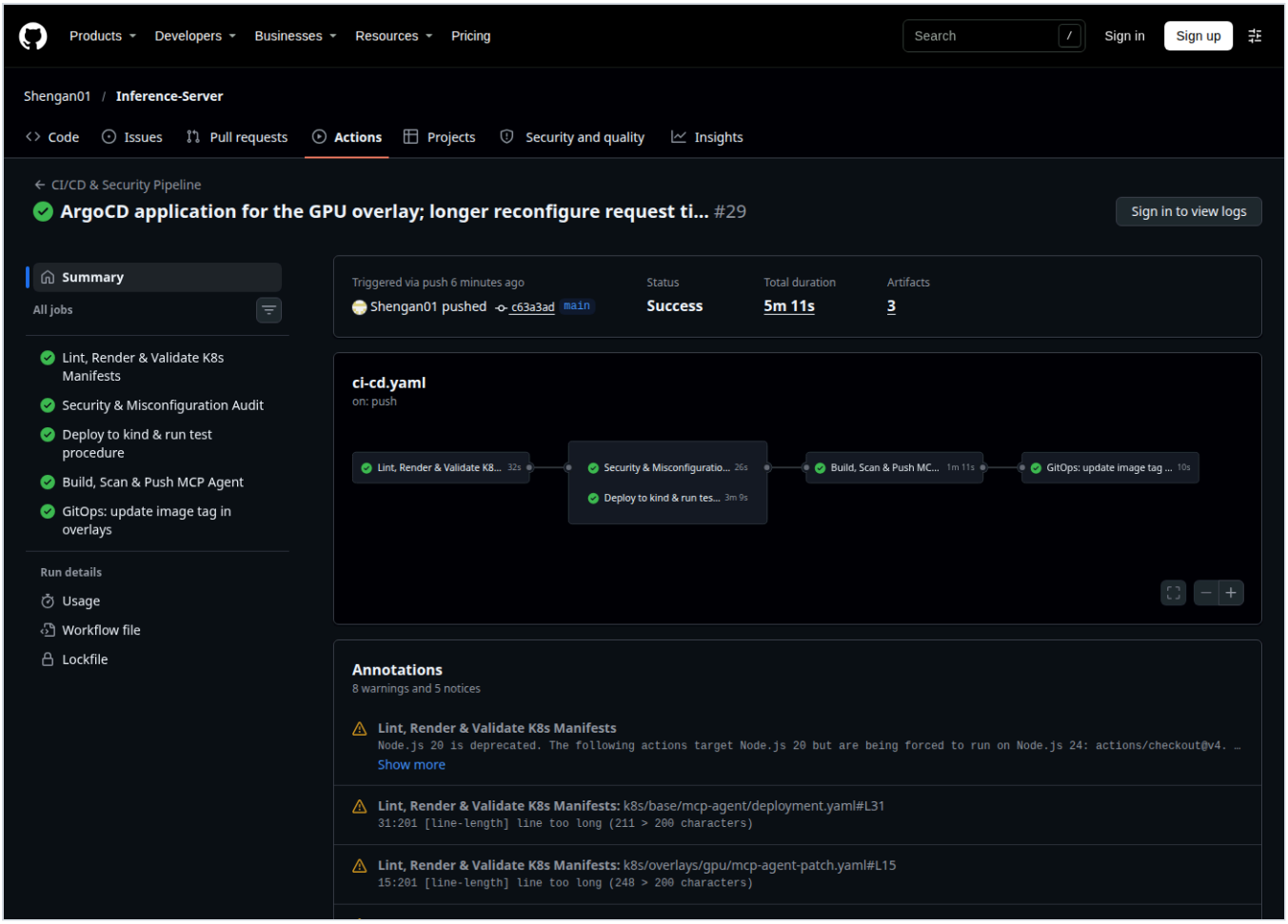
Wynik: Oba ServiceMonitory (`triton`, `mcp-agent-config`) mają stan `up`, grupa `inference_alerts` zawiera 4 reguły.

Test 8: potok CI/CD (GitHub Actions)

Cel: każdy push uruchamia walidację, skan, test w klastrze, zbudowanie obrazu i aktualizację wdrożenia (Zadanie 2, Test 8).

```
https://github.com/Shengan01/Inference-Server/actions
```


Products
Developers
Businesses
Resources
Pricing
Search
/
Sign in
Sign up
Shengan01 / Inference-Server
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Actions
All workflows
CI/CD & Security Pipeline
Management
Caches
Lockfile
All workflows
Showing runs from all workflows
Filter workflow runs
29 workflow runs
Workflow
Event
Status
Branch
Actor
ArgoCD application for the GPU overlay; longer reconfi...
CI/CD & Security Pipeline #29: Commit c63a3ad pushed by Shengan01
main
6 minutes ago
5m 11s
GPU profile: real Triton + vLLM on the host GPU
CI/CD & Security Pipeline #28: Commit 1bd8709 pushed by Shengan01
main
17 minutes ago
5m 7s
docs: task 3 report wording (results without PASS labels...
CI/CD & Security Pipeline #27: Commit 04f6c96 pushed by Shengan01
main
48 minutes ago
5m 12s
docs: task 3 report in A/B format with command output...
CI/CD & Security Pipeline #26: Commit 7a98f5b pushed by Shengan01
main
Today at 10:37 PM
4m 54s
ci: exclude placeholder example manifests from the pla...
CI/CD & Security Pipeline #25: Commit 9a1fa1b pushed by Shengan01
main
Today at 10:26 PM
5m 18s
restore mock Triton script and apply metric fixes (GPU ...
CI/CD & Security Pipeline #24: Commit f8324d0 pushed by Shengan01
main
Today at 10:19 PM
4m 6s
mock Triton: report GPU utilization as a fraction and ex...
CI/CD & Security Pipeline #23: Commit dafcebb pushed by Shengan01
main
Today at 10:18 PM
2m 32s
manifests: explicit requests/limits for init container, PV...
CI/CD & Security Pipeline #22: Commit fc3fcf2 pushed by Shengan01
main
Today at 10:04 PM
2m 32s
docs: add project links (repository, GitHub Actions, re...
main
Nov 1, 11:03 AM GMT+1



Wynik: Przebiegi na gałęzi `main` kończą się sukcesem (5 jobów: walidacja manifestów, skan bezpieczeństwa, test na `kind`, zbudowanie i skan obrazu, aktualizacja tagu w Git). Na liście przebiegów widać też trzy wcześniejsze, nieudane przebiegi z dnia poprawek (przyczyna: sprawdzenie jawnych haseł w manifestach traktowało wzorcowy plik `k8s/examples/secrets.example.yaml` jako wyciek); po naprawie wszystkie kolejne przebiegi są zielone.

3. Komentarz do funkcjonalności (zakładana i osiągnięta)

Założenia. Zbudować w Kubernetes system obserwowalności (Prometheus + Grafana + własny eksporter) dla serwera inferencji, z alertami, bezpiecznym dostępem, automatyzacją CI/CD i GitOps. Model nie jest przedmiotem oceny, jest obciążeniem generującym metryki.

Osiągnięte. Wszystkie wymagane obiekty (rozdz. 1.1) są skonfigurowane i działają, a testy 1–8 kończą się wynikiem pozytywnym. System reaguje na przeciążenie (alert), a agent SRE przywraca opóźnienie do wartości bazowej bez przestoju. Dodatkowo wdrożono profil Prometheus Operator, GitOps i potok CI/CD z testem na `kind`.

Ograniczenia (uczciwie): * **Opóźnienie na dashboardzie.** Panel *Request Latency* i alert `HighInferenceLatency` korzystają z liczników `nv_inference_request_duration_us`, które dla modelu „decoupled” (vLLM) nie mierzą czasu generowania, więc przy prawdziwym silniku pokazują wartości bliskie zeru. Wiarygodny pomiar opóźnienia daje test (end-to-end w klastrze); panele GPU, przepustowości i kolejki działają poprawnie. Pełniejsze metryki wymagałyby nowszego obrazu Triton z metrykami vLLM. * **Dobór parametrów testu.** Silnik startuje od zachowawczego `max_num_seqs = 4`, bo przy tak małym modelu (1,5B) na RTX 4080 wąskim gardłem przy większych wartościach jest narzut Pythona w backendzie vLLM, a nie pojemność batcha. Przy `max_num_seqs` = 32 różnicy

między fazami nie było. Wynik pokazuje więc poprawność mechanizmu (zmiana `model.json` → przeładowanie silnika) i jego wpływ w sytuacji, gdy batch jest ograniczeniem. * **CI bez GPU.** Runner GitHub Actions nie ma GPU, dlatego test w CI (`kind`) używa profilu `cpu-demo` z mockiem Tritona; ten sam test uruchomiony lokalnie z `TEST_PROFILE=gpu` działa na prawdziwym silniku. * **Wdrożenie GPU wymaga przygotowania hosta:** sterownik NVIDIA, runtime NVIDIA w Dockerze oraz pliki modelu w `models/finance-qwen/1/` (nie są w repozytorium). Zainstalowany `nvidia-container-toolkit` 1.12.1 ulega awarii przy `NVIDIA_DRIVER_CAPABILITIES=all`, które minikube przekazuje dla `--gpus`, dlatego skrypt `deploy_minikube_gpu.sh` używa tymczasowej podmiany tej wartości na `compute,utility`. * Alertmanager nie ma skonfigurowanego odbiorcy (e-mail/webhook); alerty są widoczne w jego interfejsie. W teście aktywny jest alert `HighInferenceQueueDepth`. * TLS jest self-signed, a sekrety tworzone są skryptem (poza Git). W środowisku produkcyjnym należy użyć cert-manager i Sealed/External Secrets. * W Minikube kontroler Ingress jest usługą `NodePort` pod `minikube ip` (w chmurze otrzymałby adres LoadBalancera). Wolumen modelu to `hostPath` (jeden węzeł). * Podczas pracy wykryto i poprawiono niezgodność mocka z prawdziwym Triton (`nv_gpu_utilization` jako ułamek 0–1, metryka `nv_inference_pending_request_count`) oraz to, że agent zmieniał tylko `config.pbtxt`, którego vLLM nie czyta (teraz aktualizuje też `model.json`).

Część B

4. Repozytorium z manifestami

Repozytorium: https://github.com/Shengan01/Inference-Server (publiczne, gałąź `main`)

Plik `README.md` w repozytorium opisuje architekturę, szybki start, strukturę katalogów, metryki i alerty oraz zawiera spis dokumentacji (w tym instrukcję uruchomienia i testów oraz wersje PDF w `docs/pdf/`).

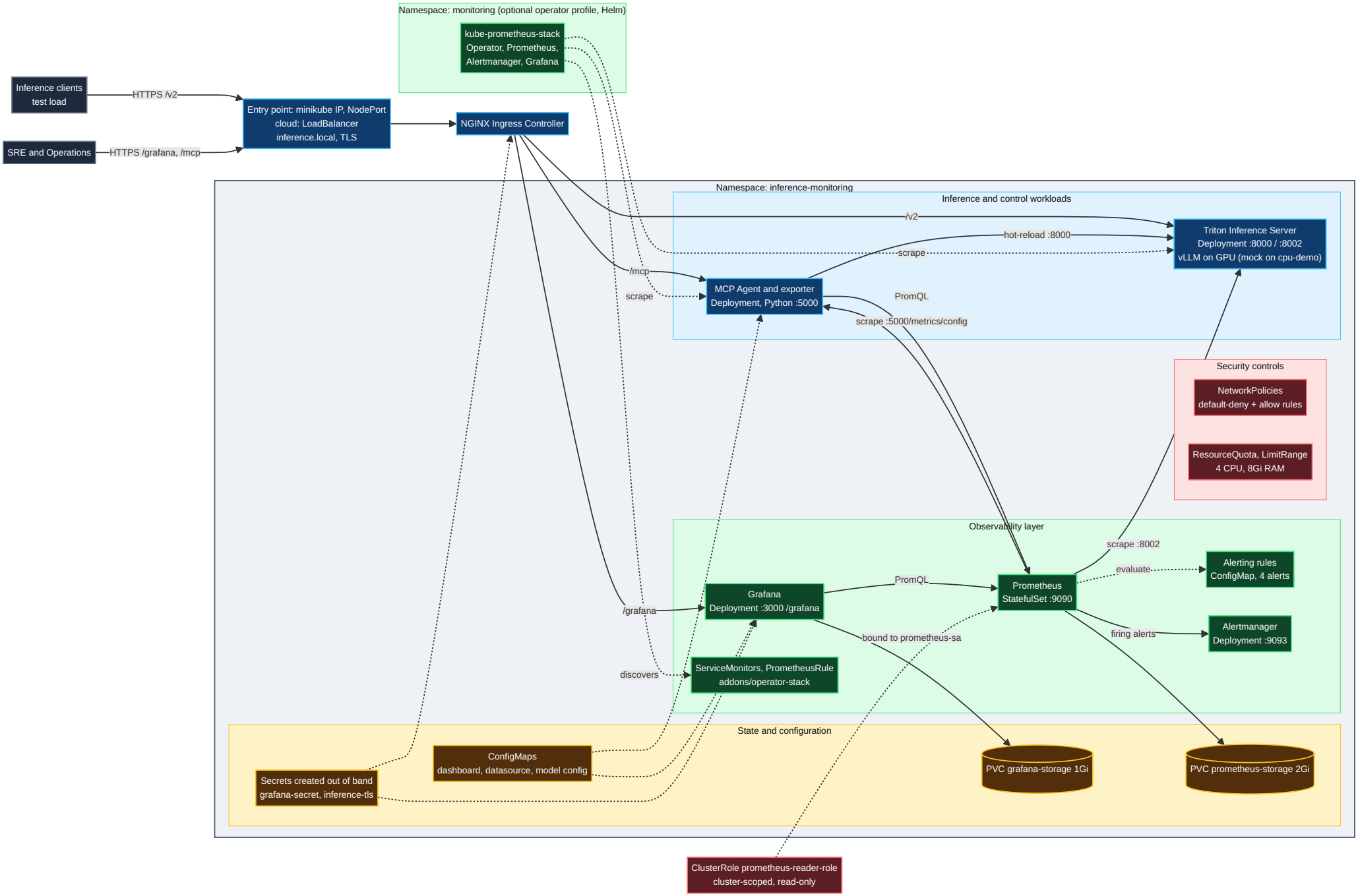
Ścieżka	Zawartość
`k8s/base/`	wspólne manifesty: namespace, RBAC, ResourceQuota/LimitRange, NetworkPolicy, Prometheus (StatefulSet), Grafana, Alertmanager, Ingress, agent
`k8s/overlays/`	profile Kustomize: `cpu-demo` (mock Triton), `gpu` (Triton + vLLM na GPU, PV z modelem), `local-cpu`, `local-gpu`
`k8s/addons/operator-stack/`	ServiceMonitory, PrometheusRule, NetworkPolicy i wartości Helm (profil operatora)
`k8s/gitops/`	aplikacje ArgoCD
`k8s/tests/`, `k8s/examples/`	Job generatora obciążenia z NetworkPolicy; wzorcowe manifesty Secret
`mcp_agent/`	własny eksporter metryk i kontroler (Python, FastMCP + FastAPI)
`scripts/`	wdrożenie (CPU i GPU), sekrety, audyt bezpieczeństwa, test w klastrze, zrzuty, PDF
`.github/workflows/ci-cd.yaml`	potok CI/CD
`docs/`	dokumentacja i sprawozdania (Zadanie 2 i 3, dokumentacja techniczna, instrukcja) oraz `docs/pdf/`

Odtworzenie wdrożenia (szczegóły w instrukcji):

```
git clone https://github.com/Shengan01/Inference-Server.git && cd Inference-Server
./scripts/deploy_minikube_gpu.sh                # klaster z GPU (wymaga plików modelu w
models/finance-qwen/1/)
TEST_PROFILE=gpu LOADGEN_IMAGE=mcp-agent:local ./scripts/run_in_cluster_test.sh
# bez GPU: ./scripts/deploy_minikube.sh cpu-demo (mock Triton) i test bez TEST_PROFILE
```

Załącznik: architektura systemu

Architektura klastra:



Dostarczanie: CI/CD i GitOps:

